

Conditional Variable and Semaphores for Threads synchronization

1. Condition Variable

A **condition variable** is used when a thread wants to **wait for something to happen** before continuing.

It always works **together with a lock (mutex)**.

How it works

- A thread **takes the lock**.
- It checks a condition (example: "Is the queue empty?").
- If the condition is **not met**, the thread:
 - goes into **wait()**
 - **releases the lock**
 - goes to **sleep**
- Another thread **signals** that the condition is now true.
- The waiting thread wakes up → **automatically takes the lock again** → continues work.

Why we use condition variables?

- To **avoid busy waiting**
(busy waiting = keeping the CPU busy by checking repeatedly).

Example

You go to a doctor's clinic:

- The nurse gives you a token and asks you to **wait**.
- You sit quietly (not busy waiting).
- When your number comes, the nurse **calls you** (signal).
- You wake up from waiting and go inside (continue execution).

2. Semaphore

A semaphore is also a **synchronization method**.

It is simply an **integer value** that tells how many resources are available.

Types of Semaphores

A) Binary Semaphore (0 or 1)

Works like a **lock**.

- 0 → resource is busy

- 1 → resource is free

B) Counting Semaphore

Used when a resource has **multiple instances**.

Example:

A parking lot with **5 parking slots** → counting semaphore = 5.

Every time a car enters → semaphore--

Every time a car leaves → semaphore++

Difference between Mutex and Semaphore (Simple)

Mutex	Semaphore
Only one thread allowed	Multiple threads allowed (depending on count)
Works like a single key	Works like multiple tokens
Ownership (only owner can unlock)	No ownership rules

Avoiding Busy Waiting

Originally, wait() would check continuously (busy waiting).

To fix this:

- If semaphore value ≤ 0 → **block the process**
- OS puts it in a **waiting queue**
- When signal() happens → OS **wakes up** one process

So CPU is not wasted.

Semaphore Example

Example 1: Bathroom with 3 doors

Counting semaphore = 3.

- 3 people can enter together.
- If all are full → next person waits.
- When one person comes out → semaphore++, someone new can enter.

wait(semaphore); // decreases count

// use bathroom

signal(semaphore); // increases count

Example 2: Binary Semaphore = Mutex

- Only one person can use the printer.
- Binary semaphore ensures **one at a time**.

What happens internally?

When wait() is called:

- If semaphore > 0 → thread continues
- If semaphore = 0 → thread is **blocked**

When signal() is called:

- One blocked thread is **woken up** (wakeup operation)
- It goes to **ready queue**

InShort

Condition Variable

- Used when thread needs to **wait for some condition**.
- Works with **mutex**.
- Avoids **busy waiting**.
- Thread sleeps → wakes up when signaled.

Semaphore

- Integer representing number of resources.
- Controls **multiple threads**.
- Two types: **Binary** and **Counting**.
- OS blocks and wakes threads → no busy waiting.